
Chapter 1: Introduction



TOPIC 1: INTRODUCTION

❑ Introduction

- Mobility
- Mobile Computing
- Mobile device
- Mobile application development platforms
- Mobile application trends and opportunity.

❑ Mobile Application Development

❑ Android Development Environment

Introduction

- A mobile device is a portable, handheld computer that enables wireless communication and general computing tasks. It is not only simple **phone call** and simple **website** access.
- In the **smart-phone** world, simply accessing a website is not enough.
- A **recent study**, has shown that more than **57%** of Google search happens using smart-phones.
- The number is spectacular and there is very rapid growth in mobile oriented business.
- Being accessible on an **Internet-enabled device** is needed for each and every **business** which has given the kicking start to mobile application development.

Introduction

- A **mobile operating** system is an operating system that helps to run other application software on mobile devices.
- Mobile apps are majorly developed for 3 Operating System.
 - **Android**
 - **IOS**
 - **Windows**
- Other
 - **Symbian**
 - **BlackBerry**
 - **Harmony**

Major Mobile Development Platforms

- IOS

- IOS is kind of arguably the **big dog** once it comes to major app development platforms.
- Came with the new paradigm.
- IOS is developed by Apple, and it runs exclusively on Apple products.
- Has good market penetration still.

- Android

- Android is the **most dominant** player in both development and market.
- It is a bit later participant to the game.
- Android first released in September 2008, virtually a year later than IOS but it has managed to achieve a reasonably **massive** share of the mobile market.
- Technically, android is the mobile OS with the largest and most dominant share of the market with around **80%** share compared to iOS's **18%** share.

Differences between iOS and Android

▪ *Android*

- Android is developed by [Google](#).
- Anyone can build an android application, and it is designed to run on a [variety](#) of different hardware platforms & devices with very different form factors and capabilities like [Samsung](#), [Huawei](#).
- Android is based on the Linux kernel, and Google releases the source code for Android developers as open source.
- Language
 - [Java](#), [Kotlin](#)

▪ *iOS*

- iOS is developed by [Apple](#).
- Operating system used on Apple devices (e.g. iPhone, iPad, iPod touches)
- Apple has limited many features of its platform and source code
- Language
 - [Swift](#), [Objective C](#)

Mobile Computing

- Mobile computing can be defined as a **computing environment over physical mobility**.
- It is a computing environment that connects a **mobile device to network or another computing device** anytime & anywhere.
- The user of the mobile computing environment will be able to access data, information or logical objects from any device in any network while on move.
 - Examples of wireless networks include **cell phone networks, WLANs, wireless sensor networks, satellite communication networks, microwave networks...**
- Mobility: is that someone / something can move or be moved easily from place to place.
- Computing: is an operation performed on computer/mobile.

...Cont'd

A computing environment is considered as mobile if it supports one or more of the following

- characteristics:

- User mobility**: user should be able to move from one physical location to another location and get

- the same service.

- Network mobility**: user should be able to move from one network to another network and get the

- same service.

- Device mobility**: user should be able to move from one device to another and get the same service.

- **Session mobility**: a user session should be able to move from one user-agent environment to another.
 - **Service mobility**: user should be able to move from one service to another easily.
 - **Host mobility**: user should be either a client or server (and host might be movable).
 -

...Cont'd

Mobile computing functions can be logically divided into the following major segments.

- **User with device**: this could be the user with a portable device like mobile phone.
 - **Network**: whenever a user is mobile, he will be using different networks at different places & at different time.
 - **Gateways**: this is required to interface different transport bearers.

Reading Assignment

- Pervasive computing
 - Ubiquitous computing
 - Global system for mobile communication and its main components
 - Switching system,
 - Base station system, and
 - Operation and support system
 -

Mobile Application Development in Android



- **Uniqueness**
stands out from other similar apps (unique design or user experience)
- **User friendly**
app is easy to use and understand, even for people who are not tech-savvy.
- **Rich features**
offers a wide range of functionality and capabilities.
- **Flawless coding**
quality of the code (free from errors and bugs)
- **Powerful marketing**
effective strategies and tactics that are used to promote and advertise a mobile app

What is Mobile Application Development?

- Mobile application development is the process of creating software applications that run on a **mobile device**.
- A typical mobile application utilizes a network connection to work with remote computing resources.
- Hence, mobile application development process involves;
 - Creating installable software bundles (code, binaries, assets, etc.),
 - Implementing backend services such as data access with an API, and
 - Testing the application on target devices.

Top mobile application development trends and Challenges

- “For many users, their smartphone is the first thing they check in the morning and the last thing they see at night.”
- **Current trends in mobile application development include:**
 1. **Cross-Platform Development:** Cross-platform development tools such as React Native, Xamarin, and Flutter are becoming increasingly popular among developers.
 2. **Artificial Intelligence and Machine Learning:** With the increasing amount of data generated by mobile devices, developers are leveraging artificial intelligence and machine learning algorithms to create intelligent and personalized applications.
 3. **Internet of Things (IoT):** The integration of mobile applications with IoT devices is a rapidly growing trend. Developers
 - **Two-Factor Authentication**
 - Several Internet & mobile applications like Gmail are using 2FA to verify a user’s identity during login or whenever suspicious activities occur on their account.
 - Reduce data breach.
 - **Blockchain**
 - Widely used in cryptocurrency and bitcoin in the real world.

Top mobile application development trends and Challenges

- Developing a mobile application comes with its own set of challenges. Some of the key **challenges** faced by developers include:
 - 1. Security:** Mobile devices store a significant amount of personal and sensitive information, making them a prime target for cyberattacks. Developers must take extra measures to ensure the security of their applications and protect users' data.
 - 2. Fragmentation:** With the numerous devices, operating systems, and screen sizes, mobile application developers must ensure that their applications work seamlessly across all devices.
 - 3. User Experience:** Mobile application users have high expectations when it comes to user experience. Developers must ensure that their applications are user-friendly and offer a seamless experience.

Why Android?

- Open source
- Many features
- Wide tech support
- Android is No. 1 Mobile Os in the current market !

Android Development Environment

- **System requirements:** android application development may be performed on any of the following system types:
 - Windows Vista (32-bit or 64-bit)
 - Windows 7 (32-bit or 64-bit)
 - Windows 8/10... / 32-bit or 64-bit
 - Mac OS X 10.8.5 or later (Intel based systems only)
 - Linux systems with version 2.15 or later
 - Minimum of 2GB of RAM (4GB is preferred)
 - Approximately 4.5GB of available disk space

...Cont'd

- Configuring a computer system
 - Install Java Development Kit (JDK)
 - Install Android Studio Integrated Development Environment (IDE) which also includes the Android Software Development Kit (SDK).
 - Create Virtual devices

...Cont'd

- Installation of Java:
 - Java is developed by Oracle Inc.
 - There are basically two Java packages:
 - Java Runtime Environment (JRE)
 - Java Software Development Kit (JDK).
 - JRE is used for running software written in Java programming language whereas JDK is utilized for developing Java software.
- Visit: <https://www.oracle.com/downloads/>

...Cont'd

- Installation of Android Studio and Android SDK
 - Android Studio is bundled with Android Software Development Kit (SDK).
 - Visit: <https://developer.android.com/studio/index.html>

...Cont'd

- Installation of Emulators
 - Emulators are software's that mimics the behavior of real devices.
 - When we develop an app, we might not have all the possible devices (Android phones, tablets, etc.) available at hand, because of this, we run the apps on emulators.
 - Emulators are also called as “Android Virtual Devices (AVDs)” in Android Studio.
 - When Android Studio is first installed, there is no default AVD.
 - We need to create one before testing our apps.

...Cont'd

The screenshot displays the Android Studio IDE with the `MainActivity.java` file open. The interface is annotated with numbered callouts:

- 1**: Points to the top toolbar, specifically the 'Run' button (a green play icon).
- 2**: Points to the breadcrumb navigation bar at the top, showing the path: `MyApplication > app > src > main > java > com > example >.myapplication > MainActivity`.
- 3**: Points to the 'MainActivity.java' tab in the editor.
- 4**: Points to the 'Project' view on the left, which shows the file structure of the `app` module, including `manifests`, `java`, and `res` folders.
- 5**: Points to the 'Debug' console at the bottom, which shows the execution flow of the `onCreate` method, including steps like `performCreate`, `callActivityOnCreate`, and `performLaunchActivity`.
- 6**: Points to the status bar at the very bottom, which displays the message: `Can't bind to local 8700 for debugger (2 minutes ago)`.

The main editor window shows the following Java code:

```
package com.example.myapplication;

import ...

public class MainActivity extends AppCompatActivity
    implements NavigationView.OnNavigationItemSelectedListener {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toolbar toolbar = (Toolbar) findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
        toolbar.setTitle("android.support.v7.w...");

        FloatingActionButton fab = (FloatingActionButton) findViewById(R.id.fab);
        fab.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                // TODO: Add your handling logic here
            }
        });
    }
}
```

...Cont'd

Android Studio UI

UI	Description
1. Toolbar	Lets you carry out a wide range of actions, including running your app and launching Android tools.
2. Nav-bar	Helps you navigate through your project and open files for editing. It provides a more compact view of the structure visible in the Project window.
3. Editor Window	Where you create and modify code. Depending on the current file type, the editor can change. For example, when viewing a layout file, the editor displays the Layout Editor.
4. Tool Window Bar	Runs around the outside of the IDE window and contains the buttons that allow you to expand or collapse individual tool windows.
5. Tool Windows	Give you access to specific tasks like project management, search, version control, and more. You can expand them and collapse them.
6. Status Bar	Displays the status of your project and the IDE itself, as well as any warnings or messages.

Android App Resources Overview

- Resources are the additional files and static content code uses (bitmaps, layout definitions, user interface strings, animation instructions, etc.).
- Always externalize app resources (images, strings) from your code, to maintain them independently.
- Provide alternative resources for specific device configurations, by grouping them in specially-named resource directories.
- At runtime, Android uses appropriate resource based on current configuration.
- For example, providing different UI layout depending on screen size or different strings depending on language setting.
- Once you externalize app resources, access them using resource IDs that are generated in your project's R class.

Grouping Resource Types

- ❑ Place each type of resource in a specific subdirectory of project's **res/** directory.

```
MyProject/  
  src/  
    MainActivity.java  
  res/  
    drawable/  
      graphic.png  
    layout/  
      main.xml  
      info.xml  
    mipmap/  
      icon.png  
    values/  
      strings.xml
```

- ❑ **res/** directory contains all resources (in subdirectories): an image resource, two layout resources, mipmap/ directories for launcher icons, a string resource file.

...Cont'd

Building Apps

- The official and best IDE for Android programming is Android Studio.
- This is a fork of JetBrains' IntelliJ IDEA application — a Java IDE customized for Android development.
- Download Android Studio bundle that includes Android SDK (Standard Development Kit): tools and libraries needed for Android development. In particular, the SDK comes with a number of useful command line tools. These include:
 - *adb*, “Android Device Bridge”, a connection between your computer and device (physical or virtual). This tool is used for console output!
 - *Emulator*, runs the Android emulator: a virtual machine of an Android device.
- Recommended: Make sure that you have the SDK tools (tools and platform-tools folder) available on your computer's PATH so you can use them from the command line.
- By default, SDK is found at `/Users/$USER/Library/Android/sdk` on Mac, and at `C:\Users\%USERNAME%\AppData\Local\Android\sdk` on Windows.

Creating Android Project

- To begin your first application, launch Android Studio (takes a few minutes to open).
- From the Welcome screen, choose to “Start a new Android Studio Project”. This will open up a wizard to walk you through setting up the project.
 - The “Company domain” should be a unique domain for you.
 - Make a mental note of the project location so you can find your work later (e.g., Desktop, Documents or D:/).
 - **If you choose to “Include Kotlin Support”, the application will be created with Kotlin rather than Java.** Since we haven’t learned Kotlin yet, we’ll start with a Java example so the little code we look at is more familiar.
 - On the next screen, you will need to pick the **Minimum SDK** level that you wish to support — that is, what is the oldest version of Android your application will be able to run on?
 - **Select target API 19 KitKat (4.4)**, due to your machine’s low computational resources (processor speed, memory size) and allowing your application to run on pretty much any Android device.

...Cont'd

- On the next screen, select to start with an Empty Activity. Activities are the basic component of Android, each of which acts as a “screen” or “page” in your app.
- Stick with the default name (MainActivity) on the next screen, and hit “Finish”.
- Android Studio will take a few minutes to create your project and get set up.
- (Keep an eye on the bottom status bar to wait for everything to be finished).
- Once it is done, you have a complete (if simple) app!

...Cont'd

Running the App

- Run the app by clicking “**Play**” or “**Run**” button at the top of the IDE.
- But run Android Device ..., Android Studio: a Virtual Android Emulator.
- This virtual machine models emulates a generic device with hardware you can specify, though it does have some limitations (e.g., no cellular service, no Bluetooth, etc).
- While it has improved recently, the emulator historically does not work very well on Windows — it runs very slowly (though it is improving!).
- The best way to **speed the emulator up** on any operating system is to make sure you have enabled **HAXM (Intel's Acceleration Manager)** which allows emulator to utilize your GPU for rendering): this speeds things up considerably.
- You can usually install this through Android Studio: go to **Tools > Android > SDK Manager** to open up the SDK manager for downloading different versions of the Android SDK and other support software.
- Under “SDK Tools”, find “Intel x86 Emulator Accelerator (HAXM installer)”, check it, and hit “OK” to download. Note that you may need to do additional installation/configuration manually, see the guides (Mac, Windows).

...Cont'd

- It is also possible to run your app on a physical device. These are best for development (fastest, easiest way to test code), though you'll need a USB cable to be able to wire your device to your computer. Any device will work for this course; you don't even need cellular service (just WiFi should work).
- You will need to turn on developer options in order to install development apps on your device!
- In order to create an emulator, go to **Tools > Android > AVD Manager** to open up Android Virtual Device Manager. You can then choose “**Create Virtual Device...**” in order to launch the wizard to specify a new emulator.
- The Pixel 2 is a good choice of hardware profile. The Nexus 5X is also reasonable for an “older” device.
- For now, you'll use a system image that works, and almost on x86 (Intel) hardware. Make sure to select one that includes the Google APIs (so you have access to special Google libraries).

...Cont'd

App Source Code

The screenshot displays the Android Studio IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, Refactor, Build, Run, Tools, VCS, Window, and Help. The title bar shows 'My Application - MainActivity.java [My_Application.app.main]'. The toolbar contains icons for running, debugging, and other development tools.

The left sidebar shows the Project view with the following structure:

- Android
- app
 - manifests
 - AndroidManifest.xml
 - java
 - com.example.myapplication
 - MainActivity
 - com.example.myapplication (androidTest)
 - com.example.myapplication (test)
 - java (generated)
 - res
 - drawable
 - layout
 - mipmap
 - values
 - xml
 - res (generated)
 - Gradle Scripts

The main editor window shows the source code of MainActivity.java:

```
1 package com.example.myapplication;
2
3 import androidx.annotation.Nullable;
4 import androidx.appcompat.app.AppCompatActivity;
5
6 import android.os.Bundle;
7
8 public class MainActivity extends AppCompatActivity {
9
10     @Override
11     protected void onCreate(Bundle savedInstanceState) {
12         super.onCreate(savedInstanceState);
13         setContentView(R.layout.activity_main);
14     }
15 }
16 }
```

The right sidebar shows the Emulator view, which displays a virtual Pixel 6 Pro API 23. The emulator screen shows the text 'My Application' at the top and 'HELLO WORLD!' in the center. The bottom status bar of the emulator shows the time 11:26 and battery level.

The bottom status bar of the IDE shows the following information:

- Version Control
- Run
- TODO
- Problems
- Terminal
- Logcat
- App Inspection
- Build
- Profiler
- Event Log
- Layout Inspector

The bottom status bar also displays the build output: 'Build APK(s): APK(s) generated successfully for 1 module: // Module 'My_Application.app': locate or analyze the APK. (today 9:16 AM)'. The system clock shows 13:48, and the encoding is UTF-8 with 4 spaces.

App Source Code

In the Android view, files are organized as follows:

- **app/** folder contains our application source code
 - **manifests/** contains the Android Manifest files, which is sort of like a “config” file for the app
 - **java/** contains the Java source code for your project. This is where the “logic” of the application goes. (This is still called java/ even if it contains Kotlin code!)
 - **res/** contains XML resource files used in the app. This is where we will put layout/appearance information
- **Gradle Scripts** contains scripts for the Gradle build tool, which is used to help compile the source code for installation on a device.

...Cont'd

XML Resources

- The **res/** folder contains resource files.
- Resource files are used to define the user interface and other media assets (images, etc). for the application.
- The different kinds of resources, which are organized into different folders:
 - **res/drawable/**: contains graphics (PNG, JPEG, etc), “drawn” on screen
 - **res/layout/**: contains user interface XML layout files for the app’s content
 - **res/mipmap/**: contains launcher icon files in different resolutions to support different devices
 - **res/values/**: contains XML definitions for general constants
- Open UI XML layout file, activity_main.xml, in the res/layout/

<Button

```
android:id="@+id/btnRegister"  
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:text="Register" />
```

...Cont'd

The Android Manifest

- Manifest File `AndroidManifest.xml`, in `manifest/` folder in Android project view.
- The Manifest acts like a “configuration” file for application, specifying application-level details such as the:
 - app name,
 - icon, and
 - Permissions (`INTERNET`, `BLUETOOTH`, `CAMERA`, etc.)
- For example, you can change the displayed name of the app by modifying the `android:label` attribute of the `<application>` element.
- By default, label is a reference to another resource found in `res/values/strings.xml` file, which contains definitions for string “constants”.
- Ideally all user-facing strings—including things like button text — should be defined as these constants.

Java Activity

- The most basic component in an Android program is an Activity, which represents a single screen in the app.
- The default provided MainActivity class is an example of this: the class extends Activity
- In this class, we override the inherited `onCreate()` method that is called by the framework when the Activity starts.
- We call the super method to make sure the framework does its stuff, and then `setContentView()` to specify what the content (appearance) of the Activity should be.
- This is passed in a value from something called **R**.
- R is a class that is generated at compile time and contains constants that are defined by the XML “resource” files!
- Those files are converted into Java variables, which we can access through the **R class**.

Java Activity Source Code Example

```
import android.app.Activity;
import android.os.Bundle;

public class MainActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        //Code Goes Here ...
    }
}
```

- This activity displays a layout (defined in the `activity_main.xml` file) when it is created. The `onCreate()` method is one of the key methods in the activity lifecycle, and it is called by the Android system when the activity is first created. In this method, we call the `setContentView()` method to specify the layout that should be displayed, and we pass in a reference to the layout file (`R.layout.activity_main`).

The Dalvik Virtual Machine (DVM)

- **DVM** was runtime used by Android to run apps on mobile devices prior to Android 5.0 (Lollipop).
- It was designed to run efficiently on devices with limited memory and processing power, and it used **Just-In-Time (JIT) compilation** to convert Dalvik bytecode (the code that Android apps are compiled to) into native machine code on the fly.
- **DVM** was replaced by the **Android Runtime (ART)** in Android 5.0, which offers improved performance and additional features.
- However, Dalvik remains an important part of Android's history and development. The process of converting Java code into **dex** files, which run on Android devices, is called "**dexing**".
- Android apps are packaged into **.apk** files, which are signed and either side-loaded onto a device or uploaded to an app store for deployment.
- The Android framework code is pre-compiled on the device, but any third-party libraries included in an app will be copied into the built app, which can increase its file size. Android apps can be developed and built on any operating system.

...Cont'd

Gradle Scripts

- After writing Java and XML source code, in order to “**build**” and **run** the app, IDE handles it for us using an automated build tool called **Gradle**.
 1. Generate Java source files (e.g., **R**) from the XML resource files
 2. Compile the Java (or Kotlin) code into JVM bytecode
 3. “**dex**”the JVM bytecode into Dalvik bytecode
 4. Pack code and other assets into an **.apk**
 5. Cryptographically sign the **.apk** file to authorize it
 6. Transfer the **.apk** to your device, install, and run it!

Lab manual for Starting Mobile programming

- [Build Your First Android App in Java | Android Developers](#)

Thank You!



Questions